第 9 章：模块与包

一句话先说清：**Python 模块不是一个只用于分类的目录名，而是“首次导入时执行代码后得到的命名空间对象”；包则把多个模块组织起来，并可以提供稳定的对外入口。**

当一个脚本逐渐变长时，模块和包主要解决三类问题：把职责拆开、隔离名称、让代码能够独立复用和测试。真实项目中的价格计算、税费规则、数据库访问和 HTTP 接口，通常都会放在不同模块中，再由应用入口组合。

学完本章后，你应该能够：‘

- 区分模块、包、发行包以及模块中的成员；
- 根据场景选择 `import` 、 `from ... import ...` 和别名；
- 解释首次导入时的查找、执行、缓存和名称绑定；
- 用 `__init__.py` 和 `__all__` 表达公开 API，同时理解它们不是访问控制；
- 正确使用包内相对导入，避开导入副作用、名称遮蔽和循环导入；
- 用 `if __name__ == "__main__":` 分隔可复用逻辑与程序入口。

9.1 先建立 Java → Python 心智模型

Java 工程师最需要调整的一点是：Java 的 `import` 主要帮助编译器解析类型名；Python 的 `import` 是一条运行时语句，会寻找模块、执行模块顶层代码并绑定名字。

Python 概念	可以先类比为	不能完全等同的地方
模块 `pricing.py`	一个无状态工具类， 或一组职责相关的类	模块本身是运行时对象，可以直接拥有函数、 常量和类
包 `shop/`	Java package	Python 普通包会执行 `__init__.py` ， 也能主动组织公开 API
模块函数	`static` 方法	Python 不需要为了容纳函数而创建工具类
`import pricing`	Java 的 `import` 加类加载的一部分效果	Python 导入发生在运行时，首次导入会执行顶层语句
`_internal_name`	非公开实现细节	单下划线只是协作约定，不等于 `private`

dg8fpjj0sff40000gp/T/crossnote-f-6a5023d9915984cb99b72d17b98.html
相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

`__all__`	一份公开 API 清单	它主要影响星号导入，不会禁止直接访问其他名字

还要区分四个容易混在一起的名词：

名词	本章例子	含义
模块（module）	`pricing.py`	一个可导入的代码单元；最常见形式是 `.py` 文件
包（package）	`shop/`	用来组织模块和子包的可导入单元
成员（member）	`pricing.calculate_total`	模块对象上的函数、类、常量等属性
发行包（distribution）	通过 `pip` 安装的项目	安装单位；安装名与代码中的导入名不一定相同

本章先讨论源码如何组成模块和包。虚拟环境、`pip` 和发行包会在后续章节继续展开。

9.2 三种导入写法分别绑定了什么

理解导入语法时，不要只记写法，要看它在当前命名空间创建了哪个名字。

导入模块：来源最清楚

```
import decimal

price = decimal.Decimal("19.90")
```

`import decimal` 在当前作用域绑定名字 `decimal`，它指向模块对象。使用成员时保留 `decimal.` 前缀，调用处能直接看出 `Decimal` 来自哪里，也不容易和本地同名变量冲突。

自定义模块同理：

```
from decimal import Decimal

import pricing

total = pricing.calculate_total(
    Decimal("19.90"), 3, tax_rate=Decimal("0.06")
)
```

README README README
f7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-6a5023d9915984cb99b72d17b98.html
关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

这里保留 `pricing.` 前缀，调用处能明确看出业务函数的来源。`Decimal` 则是含义明确、使用频繁且不易与业务名称冲突的类型，所以直接导入。

导入成员：把对象直接绑定到当前作用域

```
from decimal import Decimal

price = Decimal("19.90")
```

`from decimal import Decimal` 仍然会先加载 `decimal` 模块，然后把当时的 `decimal.Decimal` 对象绑定为当前作用域中的 `Decimal`。之后可以直接写 `Decimal(...)`，代价是调用处不再显示来源。

这个绑定不是一条会自动跟随源模块重新赋值的“活动链接”。假设有如下模块：

```
# settings.py
timeout = 10

# app.py
import settings
from settings import timeout

settings.timeout = 30
print(settings.timeout)  # 30
print(timeout)           # 10
```

两个名字最初都指向整数 `10`。`settings.timeout = 30` 只重新绑定了模块属性，不会自动改写 `app.py` 中已经绑定的 `timeout`。如果导入的是列表等可变对象，各处仍可能通过共享对象观察到内部修改；关键要区分“修改对象”和“把名字重新绑定到另一个对象”。

使用别名：解决冲突或遵守成熟惯例

```
import pathlib as pl
from collections import Counter as FrequencyCounter
```

别名只是当前模块中的另一个绑定名，不会修改源模块。它适合名称冲突、名称确实过长或社区已有稳定惯例的情况。不要只为少输入几个字符创造项目私有缩写。

可以用下面的规则做选择：

可以用下面的规则做选择：

场景	建议写法
希望调用处始终显示来源	`import pricing`
少量高频、含义明确且不易冲突的成员	`from decimal import Decimal`
名称冲突或存在公认惯例	`import module as alias`
想把所有名字倒进当前作用域	不要使用 `from module import *`

星号导入会让名字来源难以追踪，还可能静默覆盖已有名字。`__all__` 虽然能限制星号导入的结果，却不能消除这种写法的维护成本。

9.3 第一次 `import` 到底发生了什么

第一次在某个解释器进程中执行 `import pricing`，可以用下面的简化流程理解：

1. Python 根据导入名，沿导入搜索路径寻找对应模块；
2. 创建模块对象，并先登记到 `sys.modules` 缓存；
3. 从上到下执行模块的顶层语句，在模块命名空间中建立函数、类和常量；
4. 顶层代码执行完成后，在当前作用域绑定名字 `pricing`。

模块会在执行完毕前进入 `sys.modules`。这样可以支持导入过程中的递归引用，但不代表循环依赖就安全：其他模块可能拿到一个尚未初始化完整的模块对象。

同一进程后续再次导入时，Python 通常直接复用缓存中的对象：

```
import importlib
import sys

import pricing

loaded_again = importlib.import_module("pricing")
print(loaded_again is pricing)            # True
print(sys.modules["pricing"] is pricing)  # True
```

这里的缓存边界是“同一个解释器进程”。重新启动 Python 会重新导入。`importlib.reload()` 可以显式重载模块，但旧对象引用、模块状态和依赖关系会让重载行为变得复杂，不应把它当成普通业务流程中的状态重置工具。

为什么模块顶层代码必须轻量

函数体在调用时才执行，模块顶层语句却会在首次导入时执行。下面是一个教学反例：

```
# bad_config.py（反例）
print("正在导入配置")
data = download_from_network()
worker = start_background_thread()
```

任何导入者第一次加载它时，都可能触发输出、网络请求和线程启动。测试收集、IDE 分析或其他工具仅仅想读取定义，也会被迫承受这些动作。

模块顶层通常只放：

- 导入语句；
- 常量和轻量配置；
- 函数与类的定义。

网络请求、数据库连接、文件写入和线程启动应放进显式函数，再由应用入口决定何时调用。这个原则常被概括为“模块应当可以安全导入”。

9.4 如何设计一个可复用模块

模块的价值不只是把大文件切成小文件，而是建立职责和依赖边界。本章的 `pricing.py` 负责价格计算，并把税额计算交给 `tax.py`：

```
example.py  ──>  pricing.py  ──>  tax.py
    │
    └──────────>  shop/
```

箭头表示“左边导入并依赖右边”。`tax.py` 不反向导入 `pricing.py`，因此底层税费规则不需要知道上层计价流程。

一个清楚的模块接口可以很小：

```
from decimal import Decimal


def calculate_total(
    unit_price: Decimal,
```

```
def calculate_total(
    unit_price: Decimal,
    quantity: int,
    *,
    tax_rate: Decimal,
) -> Decimal:
    """根据单价、数量和税率计算总价。"""
    ...
```

`*` 之后的 `tax_rate` 是仅限关键字参数，调用者必须写 `tax_rate=...`，因此不会把税率误认成数量或其他位置参数。

设计自定义模块时，可以遵循这些原则：

- 一个模块围绕一个明确职责组织，不要不断扩张成无边界的 `utils.py`；
- 公共函数使用稳定名称，并写清输入、返回值和可能抛出的异常；
- 模块之间保持清楚、尽量单向的依赖；
- 导入集中放在文件顶部，让依赖容易发现；
- 类型提示帮助阅读和静态检查，业务约束仍需在运行时显式校验。

本章代码中的 `Decimal` 类型提示不会自动阻止负数。真正维护“价格、数量和税率不能为负”这一规则的是函数体中的条件判断和 `ValueError`。

`__all__` 不是 Java 访问修饰符

`pricing.py` 声明了：

```
__all__ = ["calculate_subtotal", "calculate_total", "format_cny"]
```

它明确表达模块希望公开的名字，并决定 `from pricing import *` 会导入哪些成员。一些文档工具也会把它当作公开 API 的提示。

但 `__all__` 不会阻止直接访问其他属性。类似 `_CENT` 这样的单下划线名称，也只是表示“内部实现，请不要依赖”。Python 更依赖命名约定、文档和稳定接口，而不是 Java `public`、包可见、`private` 那样的强制访问边界。

9.5 包如何隐藏内部文件布局

模块数量增加后，可以用包继续分组。普通包通常是一个带 `__init__.py` 的目录。本章结构如下：

```
shop/
```

502x6dg8fpjj0sff40000gp/T/crossnote-f-6a5023d9915984cb99b72d17b98.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

9.5 包如何隐藏内部文件布局

模块数量增加后，可以用包继续分组。普通包通常是一个带 `__init__.py` 的目录。本章结构如下：

```
shop/
├── __init__.py
├── discounts.py
├── rules.py
└── shipping.py
```

`shop` 不只是目录。导入后，它也是一个模块对象；首次导入普通包时，Python 会执行 `shop/__init__.py`。

用 `__init__.py` 提供稳定入口

本章的包入口重新导出了两个公共函数：

```
from .discounts import apply_discount
from .shipping import calculate_shipping

__all__ = ["apply_discount", "calculate_shipping"]
```

于是包外代码可以写：

```
from shop import apply_discount, calculate_shipping
```

调用者不需要知道两个函数目前分别位于 `shop.discounts` 和 `shop.shipping`。以后调整内部文件，只要 `shop` 的公开入口保持兼容，调用代码就不必跟着变化。这和通过 facade 隐藏内部组件有相似目的，但 `__init__.py` 本身仍是会被执行的 Python 模块，不是一个 Java 类。

公开入口也不应无限扩大。每多暴露一个名字，就多承担一份兼容责任。`shop.rules.MAX_DISCOUNT_RATE` 是内部规则，因此没有从包入口重新导出。

包内相对导入中的点表示什么

`shop/discounts.py` 使用相对导入读取同包规则：

```
from .rules import MAX_DISCOUNT_RATE
```

README README README
502x6dg8fpjj0sff40000gp/T/crossnote-f-6a5023d9915984cb99b72d17b98.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

包内相对导入中的点表示什么

`shop/discounts.py` 使用相对导入读取同包规则：

```
from .rules import MAX_DISCOUNT_RATE
```

一个点表示当前包，两个点表示父包。相对导入适合表达包内部模块的关系；包外调用者通常从顶层包开始写绝对导入，例如 `from shop import apply_discount` 。

直接执行内部文件时，它通常没有已知的父包上下文：

```
# 反例：相对导入会失败
python3 shop/discounts.py
```

如果某个包内模块被设计成命令行入口，应在包的父目录使用 `python3 -m package.module` 形式运行。 `-m` 让 Python 先按导入系统定位模块，因此能建立正确的包上下文。本章的 `discounts.py` 只负责业务函数，不是命令行入口；应运行真正的应用入口 `example.py` 。

现代 Python 也支持没有 `__init__.py` 的命名空间包，适合让同一个逻辑包跨越多个目录或发行包。初学项目和大多数单仓库业务包先使用普通包，行为更直观。

9.6 Python 去哪里寻找模块

Python 会沿 `sys.path` 查找顶层导入名。搜索路径通常来自入口脚本位置或当前启动方式、标准库、当前虚拟环境的安装目录，以及环境配置添加的路径。可以直接观察：

```
import sys

for path in sys.path:
    print(path)
```

不要在业务模块里到处写 `sys.path.append(...)` 修补导入。这样的代码会依赖当前工作目录、启动顺序和机器环境，常造成“IDE 中能运行，命令行或测试中却失败”。更稳妥的做法是建立明确的包结构，从项目约定的入口启动，并在需要时把项目安装进虚拟环境。

名称遮蔽为什么让错误看起来莫名其妙

如果在脚本旁新建 `decimal.py` 、 `json.py` 或 `typing.py` ，这个本地文件可能比标准库同名模块更早被找到：

名称遮蔽为什么让错误看起来莫名其妙

如果在脚本旁新建 `decimal.py`、`json.py` 或 `typing.py`，这个本地文件可能比标准库同名模块更早被找到：

```
project/
├── decimal.py      # 意外遮蔽标准库 decimal
└── app.py          # from decimal import Decimal
```

此时 `app.py` 可能报“无法导入 Decimal”，甚至让第三方库在内部导入时失败。定位这类问题时，可以查看模块实际来自哪里：

```
import decimal

print(decimal.__file__)
```

自定义模块应避开标准库和主要依赖的常用名称。如果当前解释器已经导入了错误模块，即使改名，它仍可能留在 `sys.modules` 中；学习和排错时最直接的做法是改名后重启解释器。

9.7 `__name__` 如何区分“被导入”和“作为入口运行”

Python 会给每个模块设置 `__name__`：

- 文件被导入时，它通常是导入名，例如 `pricing` 或 `shop.discounts`；
- 文件作为程序入口运行时，它是特殊值 `"__main__"`。

因此常见入口守卫写法是：

```
def main() -> None:
    print("启动应用")


if __name__ == "__main__":
    main()
```

导入这个文件时，Python 仍会执行顶层的函数定义和 `if` 判断，但条件为假，不会调用 `main()`。直接运行时条件为真，才进入应用流程。

`main` 不是 Python 强制规定的函数名，入口守卫也不会阻止守卫之外的顶层副作用。要实现“安全导入”，既要使用守

README README README
7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-6a5023d9915984cb99b72d17b98.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

`main` 不是 Python 强制规定的函数名，入口守卫也不会阻止守卫之外的顶层副作用。要实现“安全导入”，既要使用守卫，也要把外部动作放进 `main()` 或其他显式函数。

Java 由 JVM 调用指定类的 `public static void main(String[] args)`；Python 则先执行入口模块的顶层语句，再由普通条件判断是否调用普通函数。共同的工程原则是：入口保持轻薄，业务逻辑放进可以单独调用和测试的模块。

9.8 Java 工程师最容易踩的导入陷阱

导入方式与调用方式不匹配

```
import pricing .

# calculate_total(...)          # NameError: 当前作用域没有这个名字
pricing.calculate_total(...)    # 正确: 通过模块对象访问成员
```

只有写了 `from pricing import calculate_total`，当前作用域才会直接得到 `calculate_total` 这个名字。

误以为重复导入会重置全局状态

模块对象会缓存在 `sys.modules` 中。若模块保存了可变全局状态，同一进程中的导入者通常会共享它。需要隔离的业务状态应放进对象、函数参数或测试 fixture，而不是依赖再次 `import`。

循环导入读到了“半初始化模块”

假设 `orders.py` 顶层导入 `customers.py`，而 `customers.py` 顶层又导入 `orders.py`：

```
orders.py  —>  customers.py
    ^              |
    └──────────────┘
```

执行顺序可能是：

1. `orders` 已创建并进入缓存，但只执行了一部分；
2. `orders` 开始导入 `customers`；
3. `customers` 又从缓存取到尚未初始化完成的 `orders`；
4. `customers` 读取一个还没建立的名字，导入失败。

错误通常表现为“无法从部分初始化的模块导入名称”。优先从设计上打破环：

README README README
lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-6a5023d9915984cb99b72d17b98.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

错误通常表现为"无法从部分初始化的模块导入名称"。优先从设计上打破环：

- 把双方共享的数据模型或小协议提取到第三个模块；
- 调整依赖方向，让底层规则不知道上层编排；
- 只为类型提示而导入时，可在理解延迟注解后使用 `TYPE_CHECKING`；
- 函数内部延迟导入只作为有明确理由的局部方案，不用它掩盖职责混乱。

9.9 运行本章示例

正文已经给出了理解示例所需的完整心智模型。现在可以用本地代码验证这些结论。

配套文件的职责是：

- `example.py`：应用入口，依次演示导入方式、模块对象、缓存、包和入口守卫；
- `pricing.py`、`tax.py`：展示可安全导入的模块与单向依赖；
- `shop/`：展示包入口、相对导入和内部规则；
- `Chapter09Modules.java`、`Pricing.java`：提供 Java 组织方式的最小对照。

在项目根目录（包含 `java_to_python/` 的目录）运行：

```
python3 java_to_python/chapters/09_modules/example.py
```

输出会按小节展示以下观察点：

- `math` 和 `pricing` 都是模块对象；
- 模块导入和成员导入产生不同的调用写法；
- 第二次导入 `pricing` 得到 `sys.modules` 中的同一个对象；
- `shop` 的公开入口隐藏了 `discounts.py` 和 `shipping.py`；
- 负数价格、过大折扣等边界由运行时代码检查；
- 名称遮蔽实验会在临时目录中失败，随后自动清理；
- 直接运行时 `__name__` 是 `"__main__"`，所以会调用 `main()`。

要单独对比入口守卫，进入本章目录后运行：

```
cd java_to_python/chapters/09_modules
python3 example.py
python3 -c "import example"
```

README README README
/f7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-6a5023d9915984cb99b72d17b98.html
关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

- `math` 和 `pricing` 都是模块对象；
- 模块导入和成员导入产生不同的调用写法；
- 第二次导入 `pricing` 得到 `sys.modules` 中的同一个对象；
- `shop` 的公开入口隐藏了 `discounts.py` 和 `shipping.py`；
- 负数价格、过大折扣等边界由运行时代码检查；
- 名称遮蔽实验会在临时目录中失败，随后自动清理；
- 直接运行时 `__name__` 是 `"__main__"`，所以会调用 `main()`。

要单独对比入口守卫，进入本章目录后运行：

```
cd java_to_python/chapters/09_modules
python3 example.py
python3 -c "import example"
```

第一条 Python 命令会运行完整演示；第二条只导入模块，预期没有标准输出；因为 `main()` 受到入口守卫保护，其他模块也没有导入副作用。

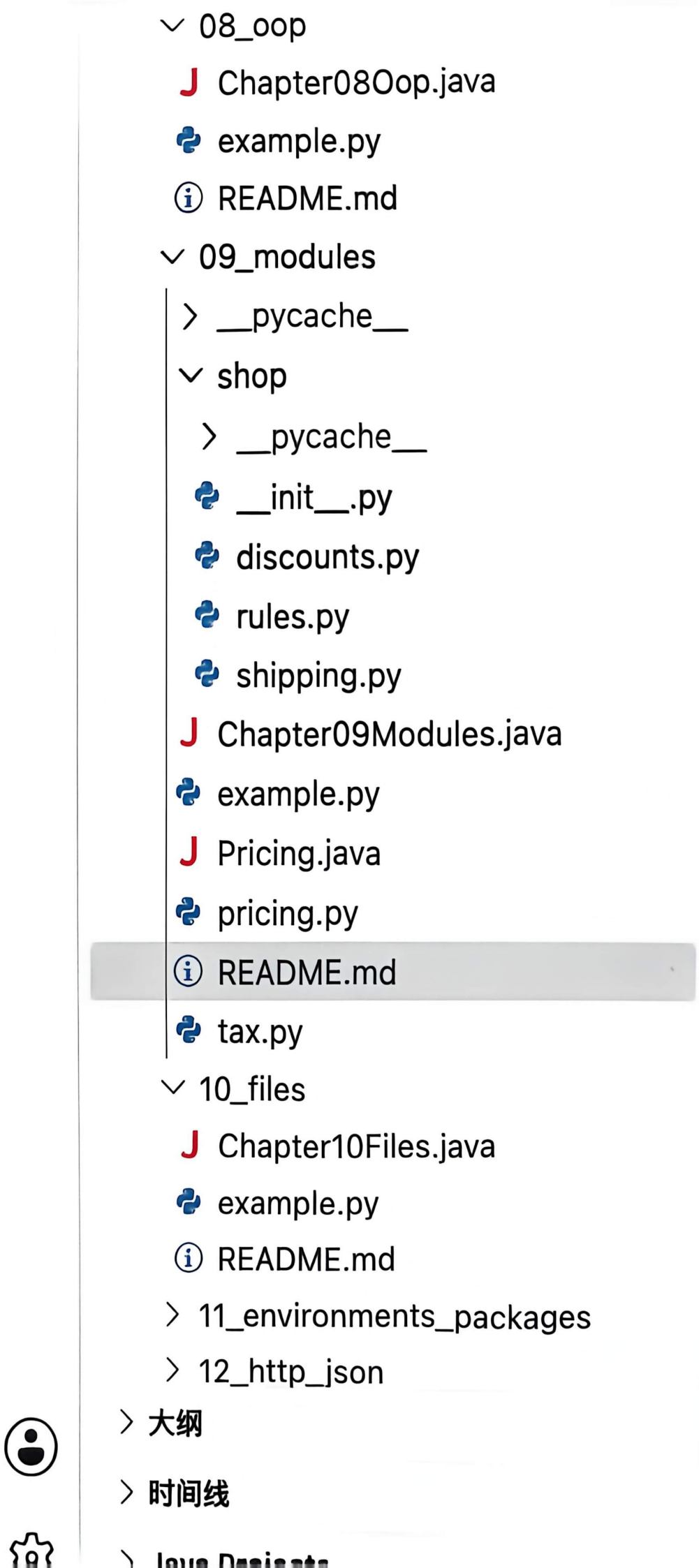
如果本机安装了 JDK，还可以在同一目录运行 Java 对照：

```
javac -encoding UTF-8 Chapter09Modules.java Pricing.java
java Chapter09Modules
```

阅读输出时，不要只比较语法长短，重点比较组织单位和启动方式：Java 把函数放进类并由 JVM 寻找 `main`；Python 可以直接在模块中定义函数，并由模块的 `__name__` 决定是否进入命令行流程。

本章小结

- `.py` 文件通常是模块；模块是运行时对象，也是独立命名空间；
- 普通包用目录组织模块，并通过 `__init__.py` 初始化和设计稳定入口；
- `import module` 绑定模块对象，`from module import name` 绑定当时取得的成员对象；
- 首次导入会查找、创建、缓存并执行模块，后续导入通常复用缓存；
- 顶层只保留轻量定义，外部动作交给显式函数和应用入口；
- `__all__` 与单下划线表达公开边界，但不提供 Java 式强制访问控制；
- 相对导入用于包内协作，包外代码通常依赖顶层包的稳定入口；
- 名称遮蔽、路径补丁、全局可变状态和循环依赖是最常见的导入问题；
- `if __name__ == "__main__"`：让同一文件既能安全导入，也能作为程序入口运行。


08_oop
Chapter08Oop.java
example.py
README.md
09_modules
__pycache__
shop
__pycache__
__init__.py
discounts.py
rules.py
shipping.py
Chapter09Modules.java
example.py
Pricing.java
pricing.py
README.md
tax.py
10_files
Chapter10Files.java
example.py
README.md
11_environments_packages
12_http_json
大纲
时间线

```python
"""商店领域包的稳定公开入口。"""

from .discounts import apply_discount
from .shipping import calculate_shipping


__all__: list[str] = ["apply_discount", "calculate_shipping"]

```

```python
"""折扣计算：演示包内模块如何通过相对导入协作。"""

from decimal import ROUND_HALF_UP, Decimal

from .rules import MAX_DISCOUNT_RATE


_CENT = Decimal("0.01")


def apply_discount(unit_price: Decimal, discount_rate: Decimal) -> Decimal:
    """返回折后单价。

    为了让示例包含可观察的错误边界．本商店最多允许五折优惠。实际项目中的规则通常来自
    配置或领域对象，而 Verbose expression in `Decimal` constructor
    """
    if unit_price < 
        raise Value
    if not Decimal("0") <= discount_rate <= MAX_DISCOUNT_RATE:
        raise ValueError(f"折扣率必须在 0 到 {MAX_DISCOUNT_RATE} 之间")

    discounted_price: Decimal = unit_price * (Decimal("1") - discount_rate)
    return discounted_price.quantize(_CENT, ROUND_HALF_UP)

```

help: Replace with `0` Ruff(FURB157)

查看问题 (F2)　快速修复... (⌥Enter)

example.py .../14_data_models × example.py .../15_protocols_generics × example.py .../16_decorators × *rules.py* 1 ×

java_to_python > chapters > 09_modules > shop > rules.py > ...

```
1  """商店折扣规则；本模块是包的内部实现细节。"""
2
3  from decimal import Decimal
4
5
6  MAX_DISCOUNT_RATE = Decimal("0.50")
7
```

1


python-quickstart-for-java
资源管理器
python-quickstart-f...
java_to_python
chapters
05_sets_dicts
example.py
README.md
06_control_flow
__pycache__
Chapter06ControlFlow.java
example.py
README.md
07_functions
__pycache__
Chapter07Functions.java
example.py
README.md
08_oop
Chapter08Oop.java
example.py
README.md
09_modules
__pycache__
shop
__pycache__
__init__.py
discounts.py
rules.py
shipping.py
Chapter09Modules.java
example.py
Pricing.java
pricing.py
README.md
tax.py
10_files
Chapter10Files.java
example.py
README.md
11_environments_packages
12_http_json
大纲
时间线
Java Projects
example.py .../14_data_models
example.py .../15_protocols_generics
example.py .../16_decorators
java_to_python > chapters > 09_modules > shop > shipping.py > calculate_shipping
"""运费规则模块；调用者应通过 shop 包入口使用公开函数。"""
from decimal import Decimal
FREE_SHIPPING_THRESHOLD = Decimal("100.00")
STANDARD_SHIPPING_FEE = Decimal("12.00")
def calculate_shipping(order_amount: Decimal) -> Decimal:
"""满 100 元免运费，否则收取 12 元。"""
if order_amount < 0:
raise ValueError("订单金额不能为负")
if order_amount >= FREE_SHIPPING_THRESHOLD:
return Decimal("0.00")
return STANDARD_SHIPPING_FEE

```python
"""第 9 章：观察模块对象、导入缓存、包和 ``__name__`` 入口守卫。"""

import importlib
import math
import subprocess
import sys
import tempfile
from collections import Counter
from decimal import Decimal
from pathlib import Path
from types import ModuleType

import pricing
from pricing import calculate_subtotal
from shop import apply_discount, calculate_shipping


def print_section(title: str) -> None:
    """让终端输出与教程小节对应，便于逐段观察。"""
    print(f"\n--- {title} ---")


def demonstrate_import_styles() -> None:
    """对比模块导入和成员导入，并展示模块是运行时对象。"""
    print_section("1. 导入方式与模块对象")

    # `import math` 保留模块前缀，调用处可以直接看出 sqrt 来自哪里。
    print("math.sqrt(81):", math.sqrt(81))
    print("math 是模块对象:", isinstance(math, ModuleType))

    # `from collections import Counter` 把成员直接绑定到当前模块。
    # Counter 高频且含义明确，直接导入能让业务表达更紧凑。
    order_statuses: list[str] = ["paid", "pending", "paid", "cancelled", "paid
    print("订单状态计数:", Counter(order_statuses))


def
```

```python

def demonstrate_custom_module() -> None:
    """调用无导入副作用的自定义模块，并观察它的命名空间。"""
    print_section("2. 自定义模块与公开 API")

    total: Decimal = pricing.calculate_total(
        Decimal("19.90"),
        3,
        tax_rate=Decimal("0.06"),
    )
    print("模块名:", pricing.__name__)
    print("公开 API:", pricing.__all__)
    print("模块导入调用小计:", pricing.calculate_subtotal(Decimal("19.90"), 3))
    print("成员导入调用小计:", calculate_subtotal(Decimal("19.90"), 3))
    print("含税总价:", pricing.format_cny(total))

    # 类型提示不会在运行时自动校验参数；真正阻止负数的是函数内的条件判断。
    try:
        pricing.calculate_total(
            Decimal("-1.00"),
            1,
            tax_rate=Decimal("0.06"),
        )
    except ValueError as error:
        print("错误路径:", error)


def demonstrate_import_cache() -> None:
    """证明重复导入通常复用 ``sys.modules`` 中的同一模块对象。"""
    print_section("3. 导入缓存")

    loaded_again: ModuleType = importlib.import_module("pricing")
    print("重复导入是同一对象:", loaded_again is pricing)
    print("sys.modules 保存该对象:", sys.modules["pricing"] is pricing)


def demonstrate_package() -> None:
```

```python
def demonstrate_package() -> None:
    """通过包的稳定入口组合折扣与计价模块。"""
    print_section("4. 包与相对导入")

    list_price = Decimal("19.90")
    discounted_price: Decimal = apply_discount(list_price, Decimal("0.10"))
    total: Decimal = pricing.calculate_total(
        discounted_price,
        3,
        tax_rate=Decimal("0.06"),
    )
    print("原单价:", pricing.format_cny(list_price))
    print("九折单价:", pricing.format_cny(discounted_price))
    print("折后含税总价:", pricing.format_cny(total))
    print("通过 shop 稳定入口计算运费:", pricing.format_cny(calculate_shipping(total)))

    # 教学反例会被窄范围捕获，因此整份脚本仍能继续执行。
    try:
        apply_discount(list_price, Decimal("0.80"))
    except ValueError as error:
        print("折扣边界:", error)


def demonstrate_readme_exercises() -> None:
    """运行模块拆分、导入副作用、包入口和名称遮蔽练习。"""
    print_section("5. README 小练习")

    print("练习 1: calculate_subtotal 已加入 pricing.__all__")
    print("  模块前缀能看出来源:", pricing.calculate_subtotal(Decimal("10.00"), 3))
    print("  成员导入调用更短:", calculate_subtotal(Decimal("10.00"), 3))

    print("练习 2: 税额模块的单向依赖")
    print("  依赖箭头: example.py -> pricing.py -> tax.py")
    print("  tax.py 不导入 pricing.py, calculate_total 复用 calculate_subtotal 和 tax")

    print("练习 3: 顶层代码、导入缓存与入口守卫")
```

```
 95  def demonstrate_readme_exercises() -> None:
107      print("练习 3：顶层代码、导入缓存与入口守卫")
108      temporary_module_name = "_chapter09_import_once_demo"
109      with tempfile.TemporaryDirectory(prefix="python-module-") as directory:
110          module_path: Path = Path(directory) / f"{temporary_module_name}.py"
111          module_path.write_text('print("  临时模块顶层代码执行")\n', encoding="utf-8")
112          sys.path.insert(0, directory)
113          try:
114              first_import: ModuleType = importlib.import_module(temporary_module_name)
115              second_import: ModuleType = importlib.import_module(temporary_module_name)
116          finally:
117              sys.path.remove(directory)
118              sys.modules.pop(temporary_module_name, None)
119          print("  连续导入复用同一对象:", first_import is second_import)
120
121      chapter_directory: Path = Path(__file__).resolve().parent
122      imported_process: CompletedProcess[str] = subprocess.run(
123          [sys.executable, "-B", "-c", "import example"],
124          cwd=chapter_directory,
125          check=True,
126          capture_output=True,
127          text=True,
128      )
129      print("  子进程只 import example 的输出:", repr(imported_process.stdout))
130      print("  当前直接运行会继续输出，因为 __name__ == '__main__'")
131
132      print("练习 4：shop 包只暴露稳定入口")
133      for amount in (Decimal("99.99"), Decimal("100.00")):
134          shipping_fee: Decimal = calculate_shipping(amount)
135          print(f"  订单 {amount:.2f} 元 -> 运费 {shipping_fee:.2f} 元")
136
137      print("练习 5：临时 decimal.py 造成标准库名称遮蔽")
138      with tempfile.TemporaryDirectory(prefix="python-shadow-") as directory:
139          shadow_path: Path = Path(directory) / "decimal.py"
140          shadow_path.write_text("# 故意遮蔽标准库，但没有 Decimal\n", encoding="utf-8")
141          shadowed_process: CompletedProcess[str] = subprocess.run(
142              [sys.executable, "-B", "-c", "from decimal import Decimal"]
```

```python
def demonstrate_readme_exercises() -> None:
    print("练习 5：临时 decimal.py 造成标准库名称遮蔽")
    with tempfile.TemporaryDirectory(prefix="python-shadow-") as directory:
        shadow_path: Path = Path(directory) / "decimal.py"
        shadow_path.write_text("# 故意遮蔽标准库，但没有 Decimal\n", encoding="utf-8")
        shadowed_process: CompletedProcess[str] = subprocess.run(
            [sys.executable, "-B", "-c", "from decimal import Decimal"],
            cwd=directory,
            capture_output=True,
            text=True,
        )
        error_lines: list[str] = [line for line in shadowed_process.stderr.splitlines() if line]
        print("  导入失败:", shadowed_process.returncode != 0)
        print("  错误末行:", error_lines[-1])
    print("  临时遮蔽文件已清理:", not shadow_path.exists())


def main() -> None:
    demonstrate_import_styles()
    demonstrate_custom_module()
    demonstrate_import_cache()
    demonstrate_package()
    demonstrate_readme_exercises()

    print_section("6. __name__ 入口守卫")
    print("当前模块名:", __name__)
    print("只有直接运行 example.py 时才会调用 main()")


if __name__ == "__main__":
    main()
```

```python
"""可复用的价格模块。

模块顶层只创建常量和函数，不打印、不读取文件，也不连接外部服务。因此其他代码可以安全地
导入本模块，而不会意外启动演示或产生外部副作用。
"""

from decimal import ROUND_HALF_UP, Decimal

import tax


__all__: list[str] = ["calculate_subtotal", "calculate_total", "format_cny"]

CENT = Decimal("0.01")


def calculate_subtotal(unit_price: Decimal, quantity: int) -> Decimal:
    """计算税前小计；该公开函数同时被 calculate_total 复用。"""
    if unit_price < 0 or quantity < 0:
        raise ValueError("价格和数量不能为负")
    return (unit_price * quantity).quantize(CENT, ROUND_HALF_UP)


def calculate_total(unit_price: Decimal, quantity: int, *, tax_rate: Decimal) -> Decimal:
    """计算含税总价，并按人民币分使用四舍五入取整。

    ``tax_rate`` 是仅限关键字参数，让调用处明确写出税率含义。类型提示服务于阅读和静态检查，
    运行时的非负约束仍由函数体显式维护。
    """
    if tax_rate < 0:
        raise ValueError("税率不能为负")

    subtotal: Decimal = calculate_subtotal(unit_price, quantity)
    tax_amount: Decimal = tax.calculate_tax(subtotal, tax_rate)
    return subtotal + tax_amount


```

```
24  def calculate_total(unit_price: Decimal, quantity: int, *, tax_rate: Decimal) -> Decimal:
27          tax_rate    是仅限关键字参数，让调用处明确与出税率含义。类型提示服务于阅读和静态检查，
28      运行时的非负约束仍由函数体显式维护。
29      """
30      if tax_rate < 0:
31          raise ValueError("税率不能为负")
32
33      subtotal: Decimal = calculate_subtotal(unit_price, quantity)
34      tax_amount: Decimal = tax.calculate_tax(subtotal, tax_rate)
35      return subtotal + tax_amount
36
37
38  def format_cny(amount:‘Decimal) -> str:
39      """把金额格式化为带人民币符号和两位小数的文本。"""
40      return f"¥{amount:.2f}"
41
```

python-quickstart-for-java

example.py .../14_data_models | example.py .../15_protocols_generics | example.py .../16_decorators | *tax.py* 1

java_to_python > chapters > 09_modules > tax.py > ...

```python
"""税额计算模块；依赖方向由 pricing 指向这里，不反向导入 pricing。"""

from decimal import ROUND_HALF_UP, Decimal


CENT = Decimal("0.01")


def calculate_tax(subtotal: Decimal, tax_rate: Decimal) -> Decimal:
    """按税前小计和税率计算税额，并四舍五入到分。"""
    if subtotal < 0 or tax_rate < 0:
        raise ValueError("税前小计和税率不能为负")
    return (subtotal * tax_rate).quantize(CENT, ROUND_HALF_UP)

```